

Exploring Deep Learning Systems Behavior by linear weight interpolation at layer level

Giorgi Merabishvili

Technical University of Munich/Fortiss

Abstract

Mimicry technique have employed layer swapping to create hybrid images by swapping feature layers from different latent vectors. However, this technique had some disadvantages, and in this report, I will discuss the advantages of new techniques. This approach offers smoother transitions between images, increased precession and overall better control over style mixing process. Additionally, these techniques minimize the distance between frontier pairs, enhancing the quality and validity of the generated images.

Switching from Layer Swapping to Linear Weight Interpolation

I have implemented linear weight interpolation instead of layer swapping which resulted in more controlled manipulations.

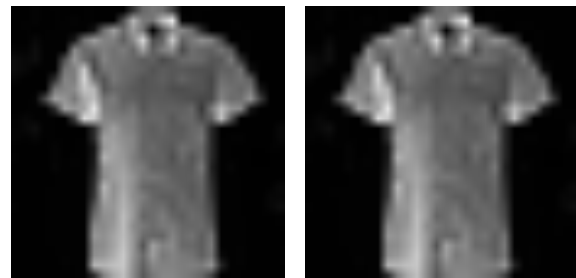
$$v = (1 - t)v_0 + tv_1$$

The adjustable alpha – t offers significant flexibility in controlling the degree of mixing between latent vectors, enabling fine-tuned and customizable style mixing. Linear weight interpolation demonstrated better results than layer swapping. While linear weight interpolation gives better results, it is computationally expensive depending on value of alpha.

Mixing Coarse Layers

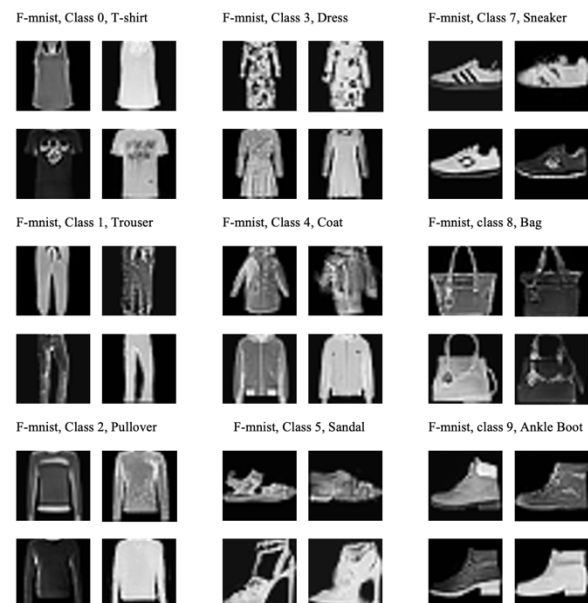
Style mixing on coarse layers such as first and second layers was not smooth with layer swapping technique. Coarse layers are responsible for overall shape and changing these feature layers introduces abrupt and unnatural transitions. Since linear weight interpolation controls the degree of mixing, it results

in seamless transitions while mixing coarse layers at low alpha. Going down on alpha makes it possible to get almost identical pairs even when mixing only coarse layers. Mixing coarse layers can be useful to generate diverse frontier pairs. Frontier pair shown on a graph 1 is a result of first layer mixing at alpha value of 0.01. L2 Distance between the frontier pair shown on graph 1 is 14 and SSIM is 0.9998.



Graph 1, F-mnist, Class 6

“Flipping” While Doing Layer Swapping on F-mnist (More Complex) Dataset



Graph 2, f-mnist, layer swapping

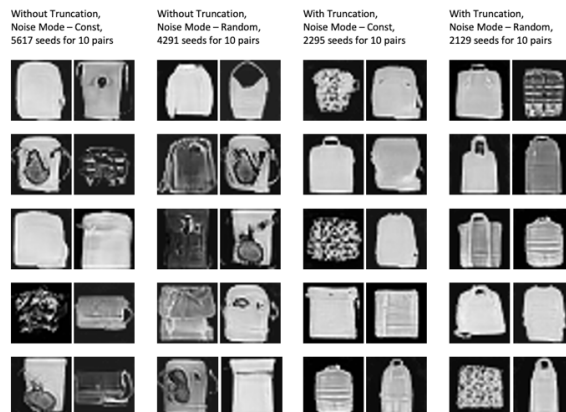
Adding linear weight interpolation solved flipping issue. At low alpha, changes during mixing process is minimal and does not affect the overall appearance even while mixing coarse layers (graph 1). Every generated pair is almost identical and impossible to distinguish by a human eye. (graph 4)

Effects of Truncation on Validity of Generated Images

Even high confidence seeds are sometimes somewhat unrealistic and unclassifiable for human eye. I found truncation helpful to increase validity of generated images. After random vector z is sampled from a normal distribution and then passed through a mapping network to produce an intermediate latent vector w , the average of w vectors is computed to get mean vector. Truncated latent vector is computed with formula, in which ψ – truncation psi has range from 0 – 1. meaning all vectors pushed to mean at $\psi = 0$ and no truncation is applied at $\psi = 1$.

$$\omega_{truncated} = mean_{\omega} + \psi \cdot (\omega - mean_{\omega})$$

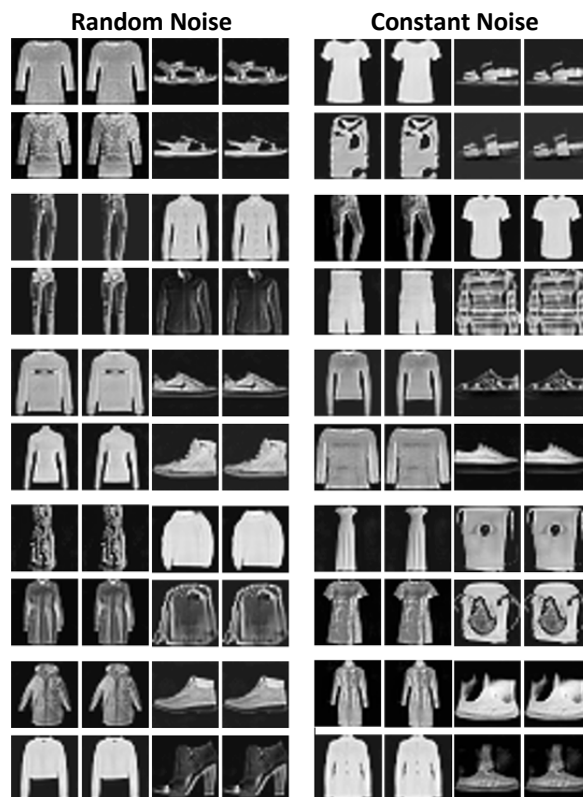
Therefore, by applying truncation we get images generated from central region of latent space which ensures to avoid unrealistic images. Optimal settings of truncation can only be found by human observation on generated results. After running various settings, I observed that PSI 0.5(middle) and truncation cutoff at layer 6 maintains both diversity and validity. After observing different settings and combinations, I found out that truncation along with random noise mode increases quality together. As shown in (graph 3, class 8, observed to be the most problematic class in f-mnist validity wise), truncation works well with random noise mode rather than const noise mode used in swapping technique. Graph shows that truncation significantly increased validity and quality of the generated images. First column shows images generated without truncation and it is obvious that these bags either have only overall shape without texture or there are two different objects on the top of each other. In the second column, while random noise helped generating better results, still transitions are not smooth. In third column, I have added truncation and constant noise which had better results than previous column. The last column is the result of truncation and random noise which has significantly better quality than the first column(base).



Graph 3, class 8

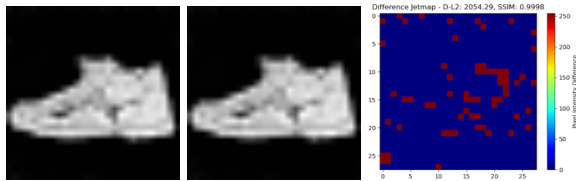
Random Noise versus Constant Noise

Based on the graph 3, setting random noise had significantly better results along with truncation. Thus, in this experiment I have tested random noise again constant noise without truncation. The graph below for both noise modes includes 2 pairs of each class from f-mnist data. Results from constant noise mode showed some unstable images, particularly in 9th class – boots and 1st class t-shirt. After running this experiment along with the experiment showed in graph 3, we can assume that random noise mode results in more stable results.



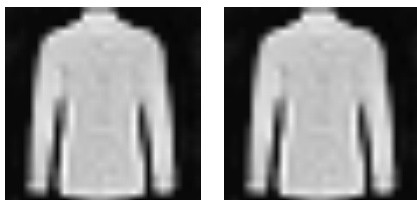
Graph 4

Effects of Background Masking

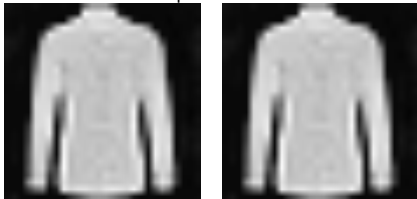


Graph 5, f-mnist, linear weight interpolation

After applying pixel difference heatmaps to frontier pairs, I observed that while generating frontier pairs using mimicry technique, either swapping or linear weight interpolation, background pixels are changed which are completely unnecessary as our main focus is the object itself and triggering the classifier by changing up the object is more useful than triggering classifier by making changes in the black background. Moreover, previous technique - DeepJanus only takes object (without background) to make changes. To solve this issue, I used masking to separate background from the object to only make changes on the object. I set manual threshold of 10, which is very loose, but it worked well. Adjusting the value of threshold might result in less changes throughout the image, leading to less L2 distance between frontier pairs. Graph 5 shows the difference between regular and masked background results. This is the first generated pair from class 6, f-mnist database. For masking manual threshold of 10 is used, which can be adjusted for possible better results. On the graph 5 (first generated image), top frontier pair (with masking) resulted in L2 distance between pair – 7.74, while bottom pair is generated without masking and resulted in L2 distance of 23.17.



F-mnist, Class 6 With Masking,
L2 distance between pair - 7.74



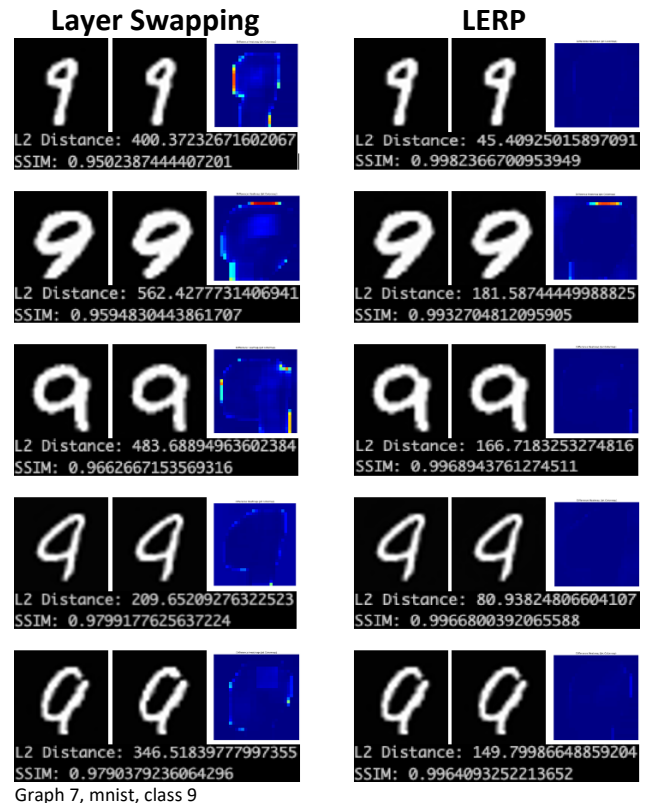
F-mnist, Class 6 Without Masking,
L2 distance between pair - 23.17
Graph 6

Metrics to Increase Validity

I have applied metrics such as Fréchet Inception Distance – FID and Perceptual hashing – pHash in validating process. After observing generated seeds and scores of pHash and FID seems like none of them can catch invalid generated images just like other metrics such as SSI and L2 distance. However, still new metrics need to be tested further to fully understand the effects of different metrics in evaluating seeds. In the most cases generated images are valid but in some cases such as class 8, f-mnist truncation is needed to generate valid images.

Comparison of Linear interpolation and layer swapping techniques on mnist database

In this experiment, there is no masking or truncation added. This is the experiment shows the difference between layer swapping and linear weight with interpolation only. The experiment was run at interpolation alpha of 0.05. Running it at lower alpha will result in lowered distance between frontier pairs but it will be computationally expensive.



This (graph 7) consists of first 5 matched seeds generated frontier pairs by both techniques. As shown, on the graph lerp resulted in 0.99... SSIM score on every generated pair and L2 distance was at least 2.5 times less than base cases, note that in the first pair LERP lowered L2 distance by almost 9 times.

Zero change to trigger classifier “Fine alpha” (Experiments mentioned before were not performed with Fine alpha)

Seeds start to be interpolated at “regular alpha” in current settings - 0.05. After it starts interpolation at every step confidence score is calculated (confidence of prediction for image being accepted, classified correctly). As confidence score drops below 1, alpha is dropped to lower value – “fine alpha” currently set to 0.0001. The goal is to have lowest alpha where confidence score is 0.5 – decision boundary. As I have “fine alpha” at the point where we trigger the classifier, we can have images very close to 0.5 confidence score. We save the images – frontier pairs, one step before triggering the classifier and the one that triggered the classifier. After, measuring the distance between correctly classified and incorrectly classified images, it turns out that L2 distance between frontier pairs is 0 and SSIM 1.0 (in most cases, if we set the fine alpha to lower, we will get perfect results in every pairs). Below is the graph that shows five first generated frontier pairs (graph 8). Every pair on the graph has been tested externally and showed L2 distance of ≈ 0 and SSIM of ≈ 1.0 .

Using advantage of swapping while interpolating

Interpolating over every seed would be very insufficient as most of them cannot be “triggered”. Therefore, before starting interpolation “Fine alpha” method performs layer swapping which maximizes chances of making changes in the image required to trigger the classifier. In this case interpolation alpha is set to 1, meaning it will fully swap the layer. After swapping the layer it checks if the confidence is still over 0.5. If it is over 0.5, it proceeds to the next stylemixing operation. If it is below 0.5 it means we can trigger the classifier and we start interpolation at

regular alpha, and as soon as confidence drops below 1, it starts to interpolate with fine alpha steps.

Acknowledgments

Distances between frontier pairs on graphs are tested externally to ensure there is no mistakes in calculation. Code can be run by the same instructions as the old technique. The difference is that you can choose any layers you want including coarse layers such as 1st, 2nd and 3rd in config.py. Also, in config.py you can adjust interpolation alpha value and based on observations truncation psi and truncation cutoff along with random noise. You can set any manual threshold on masking in search.py based on the database you are running it on, and you can set fine alpha value in search.py. Full setup is in f-mnist folder. For f-mnist, masking, interpolation, fine-alpha, random noise mode, swapping before interpolation(check) is set up and as mentioned truncation can be added from config.py if needed.

Future Work

- Interpolation method is computationally heavy, implementing non-linear search algorithm will reduce the time of running. Also, adding adaptive alpha can drop running time.
- Test different metrics in seed validation process to avoid invalid images without truncation.
- As experiment showed truncation resulted in more stabilized images (graph 3), introducing metric measuring distance between $\omega_{truncated}$ – truncated image/vector and untruncated image/vector might be useful.
- Implement automatic thresholding instead of manual thresholding that is currently used to separate background and the object. For non-black and white datasets current method will not work, thus implementing methods that can automatically separate background from the object in colorful datasets will be needed.



```
"predictions": [  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.50079345703125,  
  0.0,  
  0.0,  
  0.49920654296875,  
  0.0  
],
```



```
"predictions": [  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.49993896484375,  
  0.0,  
  0.0,  
  0.50006103515625,  
  0.0  
],
```



```
"predictions": [  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.5020370483398438,  
  0.0,  
  0.49796295166015625,  
  3.8397492096202255e-38,  
  0.0  
],
```



```
"predictions": [  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.49971771240234375,  
  0.0,  
  0.5002822875976562,  
  3.82726251926432e-38,  
  0.0  
],
```



```
"predictions": [  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.50274658203125,  
  0.0,  
  0.4972534775733948,  
  0.0  
],
```



```
"predictions": [  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.499237060546875,  
  0.0,  
  0.0,  
  0.500762939453125,  
  0.0  
],
```



```
"predictions": [  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.5031737685203552,  
  0.0,  
  0.0,  
  0.4968262016773224,  
  0.0  
],
```



```
"predictions": [  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.4974976181983948,  
  0.0,  
  0.0,  
  0.50250244140625,  
  0.0  
],
```



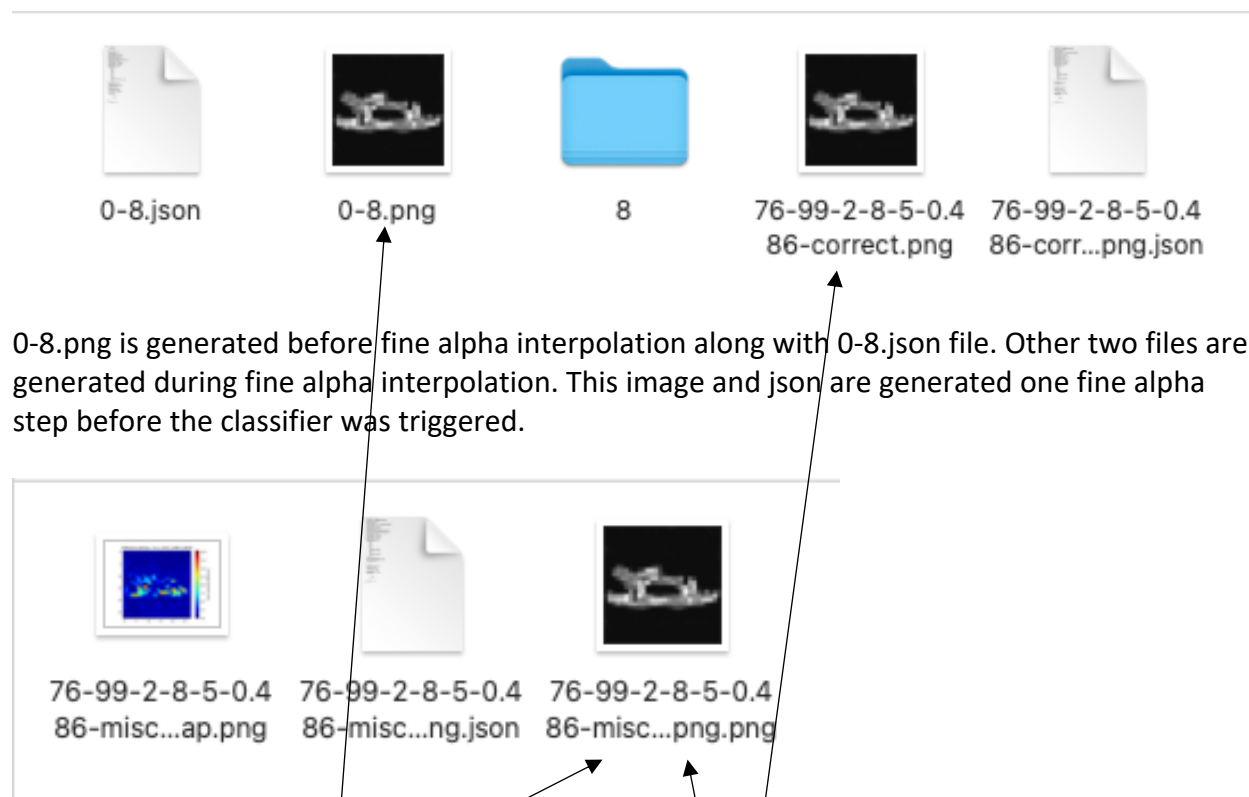
```
"predictions": [  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.5048216581344604,  
  0.0,  
  0.0,  
  0.4951784014701843,  
  0.0  
],
```



```
"predictions": [  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.0,  
  0.49548354744911194,  
  0.0,  
  0.0,  
  0.5045164823532104,  
  0.0  
],
```

graph 8, f-mnist, class 5, Fine Alpha = 0.0001

In every generated folder there are:



0-8.png is generated before fine alpha interpolation along with 0-8.json file. Other two files are generated during fine alpha interpolation. This image and json are generated one fine alpha step before the classifier was triggered.

In folder "8", we have heatmap measuring distance between image generated before fine alpha and after fine alpha image(misclassified).

Therefore, frontier pair in this case are those two images, usually scoring ≈ 0 for SSIM and L2, so adding a heatmap for those images would be useless.

